

How It Works — Key Concepts

Neural Polar Decoding for Two-User MAC

April 2026

[title=1. MAC Polar Codes] Two users U and V each independently encode their message bits using a standard polar encoder ($G_N = B_N F^{\otimes n}$: bit-reversal then XOR butterfly). Both transmit simultaneously; the receiver sees $Z = f(X, Y) + \text{noise}$. A **path** $\pi \in \{0, 1\}^{2N}$ determines the decoding order: at each of $2N$ leaf positions, the decoder decides one bit of either U or V . Class B (symmetric rate) uses an interleaved path; Class C (corner rate) decodes all of U first, then V .

[title=2. The Two Core Operations] All polar decoding reduces to two tensor operations on 2×2 probability tables (one dimension per user):

Circular convolution (circ_conv): Marginalizes over an unknown bit linked by XOR encoding. Given a tensor $T(a, b)$ representing $\Pr[\text{data} \mid u_{\text{left}} = a, v_{\text{right}} = b]$, convolution sums over the values consistent with $u_{\text{left}} \oplus u_{\text{right}} = c$:

$$(\text{circ_conv } T)[c] = \sum_{a \oplus b = c} T(a, b)$$

This is exactly circular convolution of each row/column. Used in **CalcLeft**.

Normalized product (norm_prod): Combines two independent pieces of evidence about the *same* variable. If T_1 and T_2 are both likelihoods for the same (u, v) :

$$(\text{norm_prod}(T_1, T_2))[a, b] \propto T_1(a, b) \cdot T_2(a, b)$$

This is pointwise multiplication followed by normalization. Used in **CalcRight** and at leaves.

[title=3. CalcLeft / CalcRight / CalcParent] The decoder operates on a binary tree with N leaves. Each edge stores a 2×2 tensor.

- **CalcLeft** (top-down): Given the parent’s top-down message, compute the left child’s message using **circ_conv**. This marginalizes over the right child’s unknown contribution.
- **CalcRight** (top-down): Given the parent’s top-down message and the left child’s decision, compute the right child’s message using **norm_prod**. The left child’s value is now known, so we condition on it.
- **CalcParent** (bottom-up): After deciding a leaf, propagate the decision upward. Converts a child’s “as-child” tensor to the parent’s “as-parent” tensor using **circ_conv**.

These three operations are applied recursively, navigating the tree in bit-reversal order to decode all $2N$ bits.

[title=4. Chained Decoding (Two-Stage)] For the MAC, decoding is **chained**:

1. **Stage 1:** Decode user U , treating V as noise. The channel likelihoods marginalize over all possible V values.
2. **Stage 2:** Given the decoded $\hat{U}$, subtract its contribution from the received signal and decode V .

For Class C (corner rate), the path naturally separates the two stages. For Class B (interleaved path), U and V bits are decoded alternately — each decision affects both users.

[title=5. The Neural Replacement (NPD / NCG)] Replace exact 2×2 tensors with learned d -dimensional embeddings:

- Each edge stores a vector $\mathbf{e} \in \mathbb{R}^d$ instead of a 2×2 table.
- **CalcLeft**, **CalcRight**, **CalcParent** become **learned MLPs**: $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$.
- At each leaf, a **decision head** maps the d -dim embedding to a bit probability.
- Channel observations z are mapped to initial d -dim embeddings via a learned **$\mathbf{z}$.encoder**.

NPD (Neural Polar Decoder): replaces CalcLeft/Right/Parent with neural networks.

NCG (Neural Computational Graph): same idea applied to the full SC computational graph, using the interleaved tree structure from Ren et al. (2025).

Key advantage: the neural operations can implicitly learn channel structure (noise distribution, memory, interference) without explicit channel modeling.

[title=6. Training: **fast_ce**] Training is **teacher-forced and parallel**:

- Generate random codewords and channel outputs.
- Run the full tree computation with *ground-truth* bits at each decision (teacher forcing). No sequential error propagation during training.
- Cross-entropy loss at every leaf: $\mathcal{L} = -\sum_{t=1}^{2N} \log p_\theta(\hat{b}_t = b_t)$.
- All $2N$ leaves computed in one forward pass; gradient depth is $O(\log N)$ (tree depth), not $O(N)$.

This enables training at $N = 1024$ in reasonable time. At *test* time, decoding is sequential (each decision feeds into the next).

[title=7. Three-Phase Design]

1. **Rate-1 training:** Train the neural decoder at rate 1 (no frozen bits). The model learns the channel and tree operations without needing a good frozen-set design.
2. **MI ranking:** Use the trained model's leaf cross-entropies as mutual information estimates. Rank bit positions by reliability. Select the $k_u + k_v$ most reliable positions as information bits.
3. **Retrain at target rate:** Freeze the selected positions and retrain the decoder. The frozen-set and decoder are now *co-adapted*.

This avoids the chicken-and-egg problem: you need a good frozen set to train a decoder, but you need a decoder to evaluate frozen-set quality.